Project of AI for Cybersecurity

DATA MINING WORKFLOW ON "HULK-NODEFENSE" DATASET

FABBRI ANDREA

The Goal

Train a machine learning model to accurately classify malicious internet connections involved in a "Hulk" attack, identify the key features and analyze how the model's performance changes when these features are not provided.

Specifications

- Data Collection :
  - Obtain the "Hulk-NoDefense" dataset.
- Data Exploration:
  - Analyze the dataset to understand its structure and contents.
  - Identify correlations, and potential anomalies in the data
- Data Preprocessing
  - Handle missing values, duplicates, and outliers, normalize and implement techniques for rebalancing the dataset.

- Model Training with Features Analysis
  - Conduct feature importance analysis to identify key attributes.
  - Adopt a dimensionality reduction method
- Models Evaluation
  - Evaluate different trained models using accuracy, precision, recall, F1-score
  - Compare models to select the best-performing approach.

Data Acquisition

“Hulk-NoDefense” dataset

1

The dataset comprises monitored connections of various servers during a Hulk attack or *HTTP Unbearable Load King*, that is a denial-of-service (DoS) attack method that aims to overwhelm a web server by generating numerous unique and seemingly legitimate requests to exhaust server resources.

Flow ID	Src IP	Src Port	Dst IP	Dst Port	Protocol	Timestamp	Flow Duration	Total Fwd Packet	Total Bwd packets	...	Fwd Seg Size Min	Active Mean	Active Std	Active Max	Active Min	Idle Mean	Idle Std
192.168.111.65-192.168.111.66-34700-80-6	192.168.111.65	34700	192.168.111.66	80	6	11/02/2021 09:03:53 AM	1397	7	6	...	32	0.0	0.0	0.0	0.0	0.0	0.0
192.168.111.65-192.168.111.66-34702-80-6	192.168.111.65	34702	192.168.111.66	80	6	11/02/2021 09:03:53 AM	1438	7	6	...	32	0.0	0.0	0.0	0.0	0.0	0.0
192.168.111.65-192.168.111.66-34704-80-6	192.168.111.65	34704	192.168.111.66	80	6	11/02/2021 09:03:53 AM	1753	7	6	...	32	0.0	0.0	0.0	0.0	0.0	0.0
192.168.111.65-192.168.111.66-34706-80-6	192.168.111.65	34706	192.168.111.66	80	6	11/02/2021 09:03:53 AM	3166	7	6	...	32	0.0	0.0	0.0	0.0	0.0	0.0
192.168.111.65-192.168.111.66-34708-80-6	192.168.111.65	34708	192.168.111.66	80	6	11/02/2021 09:03:53 AM	1094	9	7	...	20	0.0	0.0	0.0	0.0	0.0	0.0

2

It’s made of 851173 istances, described by 84 numerical and categorical features, that span from the source IP address, to the Active and Idle mean.

Data Exploration

Numerical Inspection

Main Step:

1. Data Description
2. Checks for NaN/inf values
3. Inspection of ‘Label’ feature
4. Inspection of always-zero features
5. Computation of chi-square on categorical values

Results:

Two ‘Label’ values: “BENIGN”, “Hulk-NoDefense”

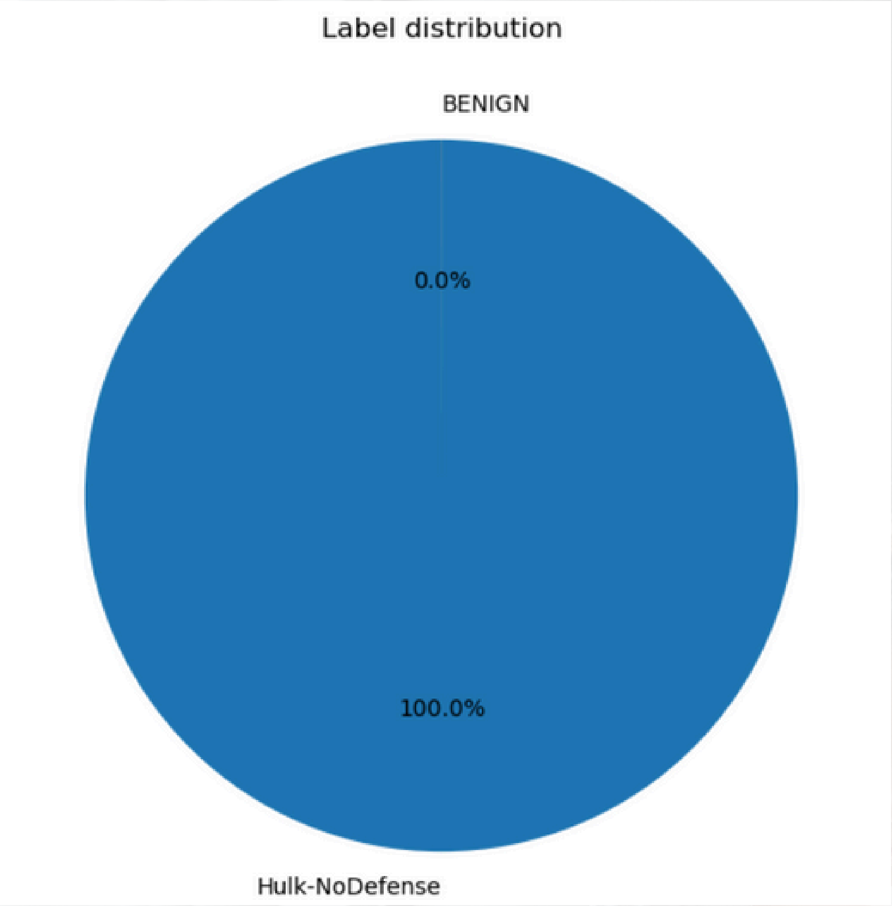
Three istances with NaN/inf values

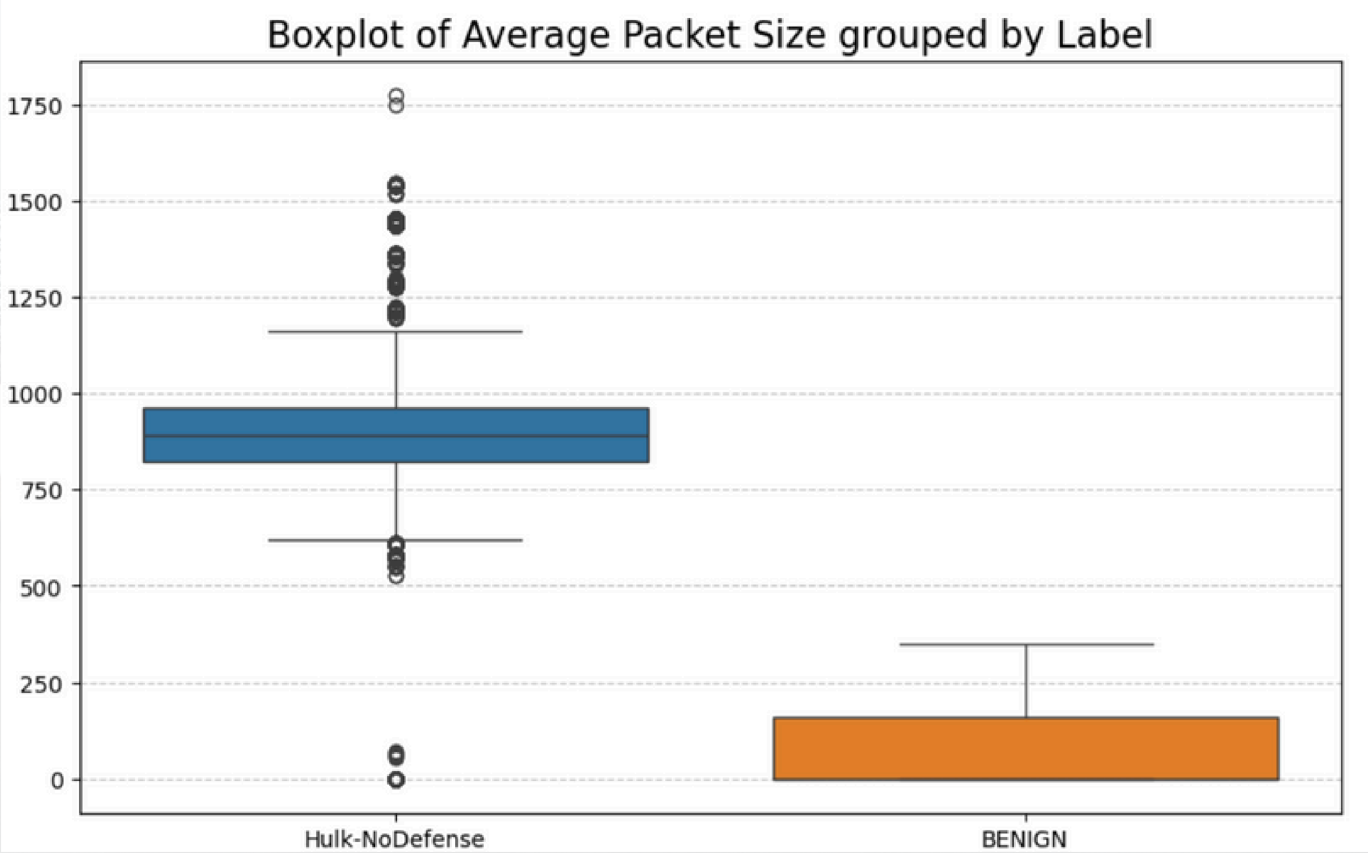
Six always-zero features

Categorical features indipendent from ‘Label’

Data Exploration

Visual Inspection





Results:

Strongly Unbalanced Dataset

Substantial differences between feature values for instances of different classes

A lot of features are positively or negatively correlated

Expectations

- Good performances with simple classifiers
- Good performances with a sub-set of features
- Rebalancing is a possible issue

Data Preprocessing

1 Label Encoding

Label values ‘**BENIGN**’ and ‘**Hulk-NoDefense**’ encoded into 0 and 1

2 Useless features deletion

Deletion of always-zero features and ‘Label’ from the training set

3 Missing/bad values management

Problem

In each case the “**Flow duration**” is 0 and there’s only 1 packet in the connection. Those values lead to have "**Flow IAT Mean**", "**Flow IAT Std**," "**Flow IAT Max**," and "**Flow IAT Min**" equal to NaN and "**Flow packet/s**" and "**Flow Byte/s**" equal to inf.

Solution

If the flow consists of only 1 packet, 0 is considered a valid value, indicating no flow. In this case, "**Flow Bytes/s**" and "**Flow Packet/s**," previously set to 'inf,' are now set to the packet's byte count and 1, respectively. Since there's no flow, '**Flow IAT Mean**', '**Flow IAT Std**,' '**Flow IAT Max**,' and '**Flow IAT Min**' are all set to 0

4 Categorical Feature Management

The remaining categorical field is ‘**Timestamp**’. From that, three numerical fields were extracted that are ‘**Hour**’, ‘**Weekday**’ and ‘**Month**’. That’s because in a field of cybersecurity and protection against Dos attack, could be relevant to understand if a particular, strange connection happens in office hour or not.

5 Scaling

All the data inside the training set are scaled, using “**StandardScaler()**”, to make the features have the same weight for the classification.

Data Preprocessing- Rebalancing

Rebalancing Techniques

1. Using only ***Random Under-Sampling*** **(RUS)**
2. Adopting an hybrid approach with ***Synthetic Minority Oversampling Technique*** **(SMOTE)** and **RUS** on numerical features, with different sampling strategies
   a. 0.01
   b. 0.008
   c. 0.005
   d. 0.0025
3. Hybrid approach with **SMOTE** and **RUS** with transformed numerical features.
4. Hybrid approach with ***Adaptive Synthetic Sampling*** **(ADASYN)**

The evaluation method chosen is **StratifiedKFold**, scoring accuracy, precision, recall and f1-score.

The General Pipeline

Implemented to avoid data leaks, it involvs:

1. The Scaler

Always the **StandardScaler**

2. Over-Sampling Method

SMOTE with the correct sampling strategy or **ADASYN**

3. Under-Sampling Method

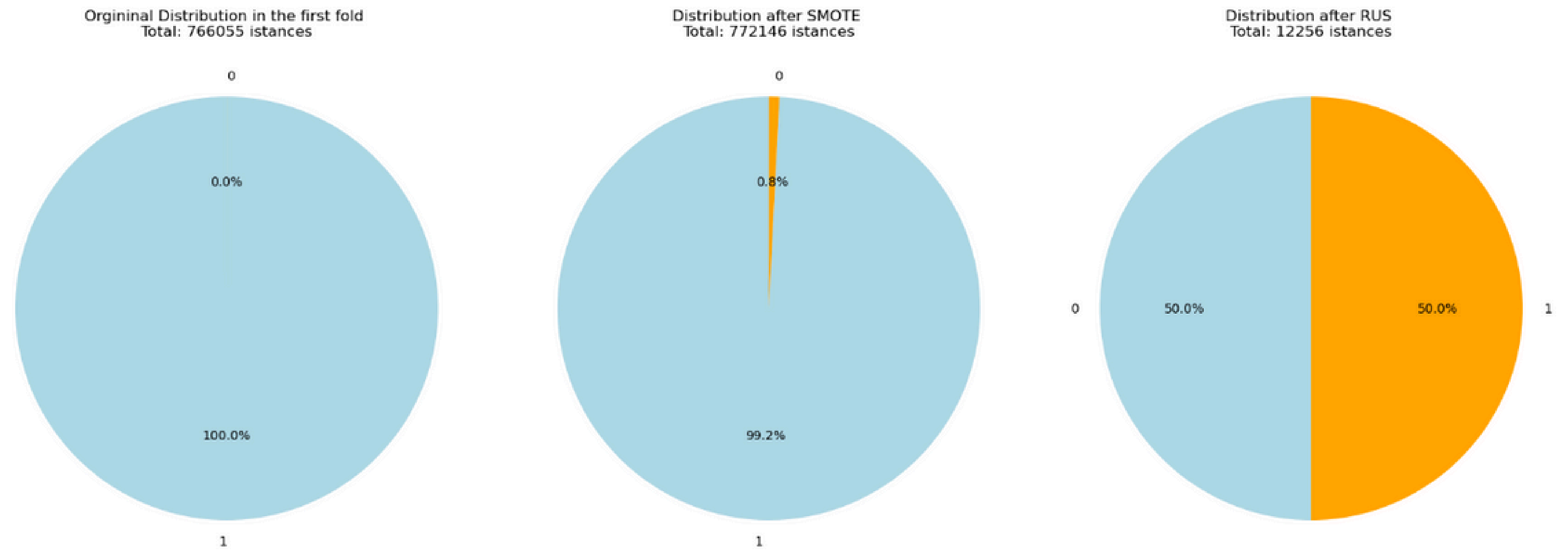
Always **RUS** with the sampling strategy equal to *majority*

4. The Classifier

The base classifier trained is **Logistic Regressor**, mainly because it's simple, easy to understand and computationally efficient.

Data Preprocessing- Rebalancing

Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1
1.0	1.0	1.0	1.0	1.0	1.0	1.0	1.0



For the next phases, the base pipeline will use **SMOTE with a sampling strategy of *0.008***, *to* generate an appropriate number of synthetic samples, ensuring a sufficient amount of data after applying undersampling with RUS

Results

By plotting the distribution before and after each rebalancing technique is possible to see that the final number of istances is different, but the performances of the classifier are always perfect.

This can be attributed to the fact that many features show significantly different values between instances of the two classes, making the classification task relatively simple

Next

What are the most important features
and what happens if they are removed?

Feature Selection - Feature Importance

Objective:

Discover how the model take the decision, by looking at the feature importance, to understand how the performances are affected if some of those features are not provided

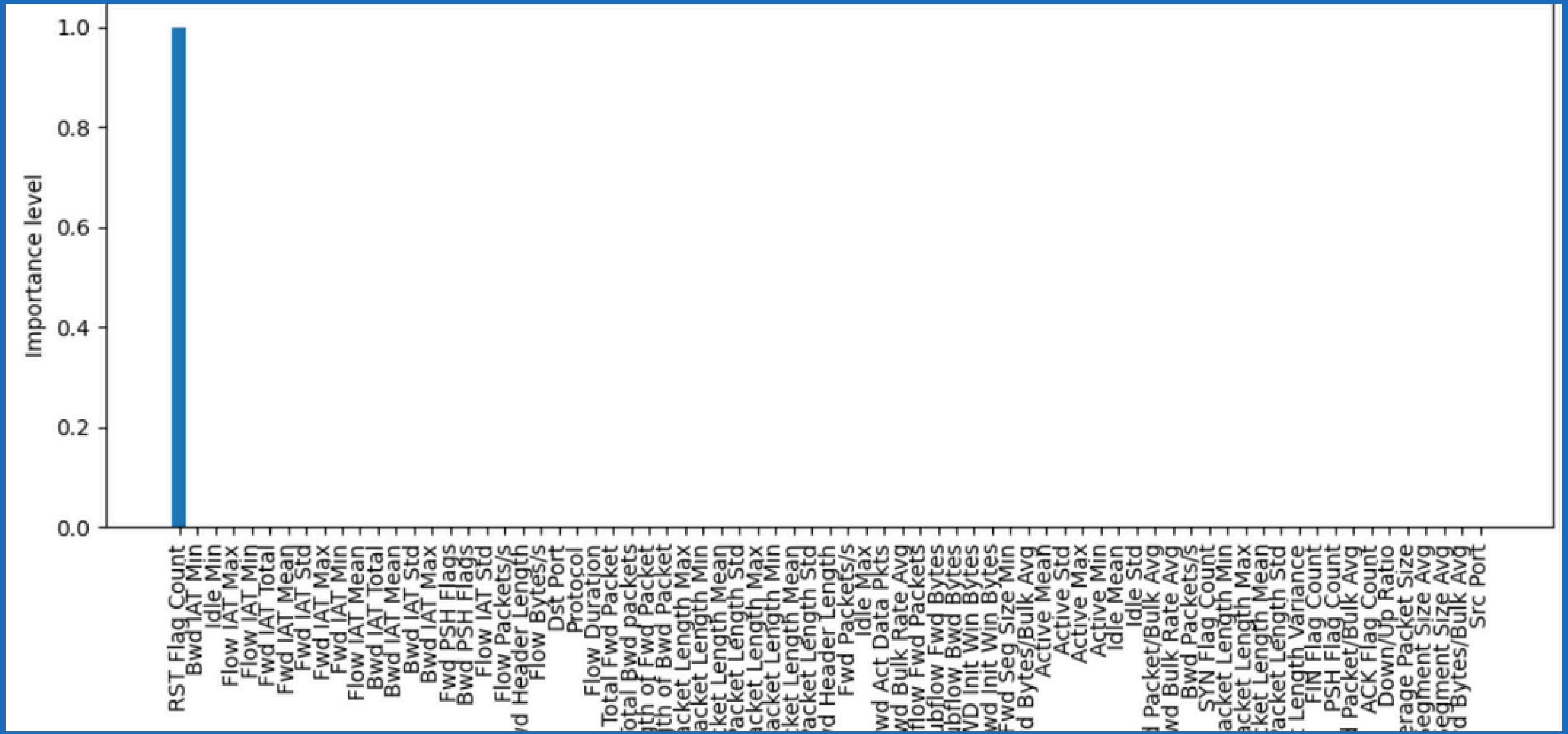
Steps:

1 Feature Importance computation

Using **DecisionTreeClassifier**, in conjuction with **SMOTE and RUS,** only on the numerical features.

The most important feature is ‘RST FLAG COUNT’, which represents the number of packets with the reset flag enabled.

This is significant because a malicious connection that generates numerous requests with a high volume of data can cause the server to attempt to reset the connection. In contrast, such behavior is typically not observed in legitimate connections.



2 Removing important characteristics

The t-test is not applicable to remove the features with a big difference in value among istances of different classes, so based on the feature importance, the most important features are removed. Specifically, features are removed until only 10, 5, and finally 1 remain

Feature Selection - Feature Importance

2 **Removing important characteristics** cont.

The results are that even with a sub-set of the least important features, the **LogisticRegressor** is able to correctly classify all of the istances, with a little loss in performances that change depending on the remaining features.

3 **Repeting the analysis with the extracted numerical features**

By adding the features '**Hour**', '**Weekday**' and '**Month**' , the performances tends to degradate, beacause the removal of the most important features causes the remaining ones to be less informative than the previous case. In fact, as resulted from the chi-sqaure analysis, those added features are not related to the target.

It's notable the fact that the performances with only five remaining features are comparable with the previous case, stating that they still carry enough information to allow the model to maintain high accuracy

Performance on the feature sets resulting from the third step

Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1	
0.31346	0.31346	0.999996	0.999996	0.313428	0.313428	0.477267	0.477263	Performance with 10 features
0.99996	0.99996	0.99996	0.99996	1.0	1.0	0.99998	0.99998	Performance with 5 features
0.000048	0.000048	0.0	0.0	0.0	0.0	0.0	0.0	Performance with 1 feature

Feature Selection - Mutual Information

Why?

To provide another perspective on feature importance, based on an heuristic approach that quantifies the statistical dependency between each feature and the target variable, assessing each feature independently.

Mutual Information identifies features with strong individual relevance, while **DecisionTree** feature selection highlights features that contribute to the overall model performance through interdependencies.

Two Main Results:

1. The most important features are different than the previous case:
    - Thanks to the differences in assessing the features
2. The performance remains high:
    - Because the dataset contains many features that, when considered in isolation, can effectively discriminate between different classes

```
The best 10 features are  Index(['FWD Init Win Bytes', 'Fwd Act Data Pkts', 'Dst Port', 'Bwd PSH Flags',
       'PSH Flag Count', 'Fwd PSH Flags', 'FIN Flag Count',
       'Total Length of Bwd Packet', 'SYN Flag Count', 'Protocol'],
      dtype='object')
The best 5 features are  Index(['FWD Init Win Bytes', 'Fwd Act Data Pkts', 'Bwd PSH Flags', 'Dst Port',
       'PSH Flag Count'],
      dtype='object')
The best feature is  Index(['FWD Init Win Bytes'], dtype='object')
```

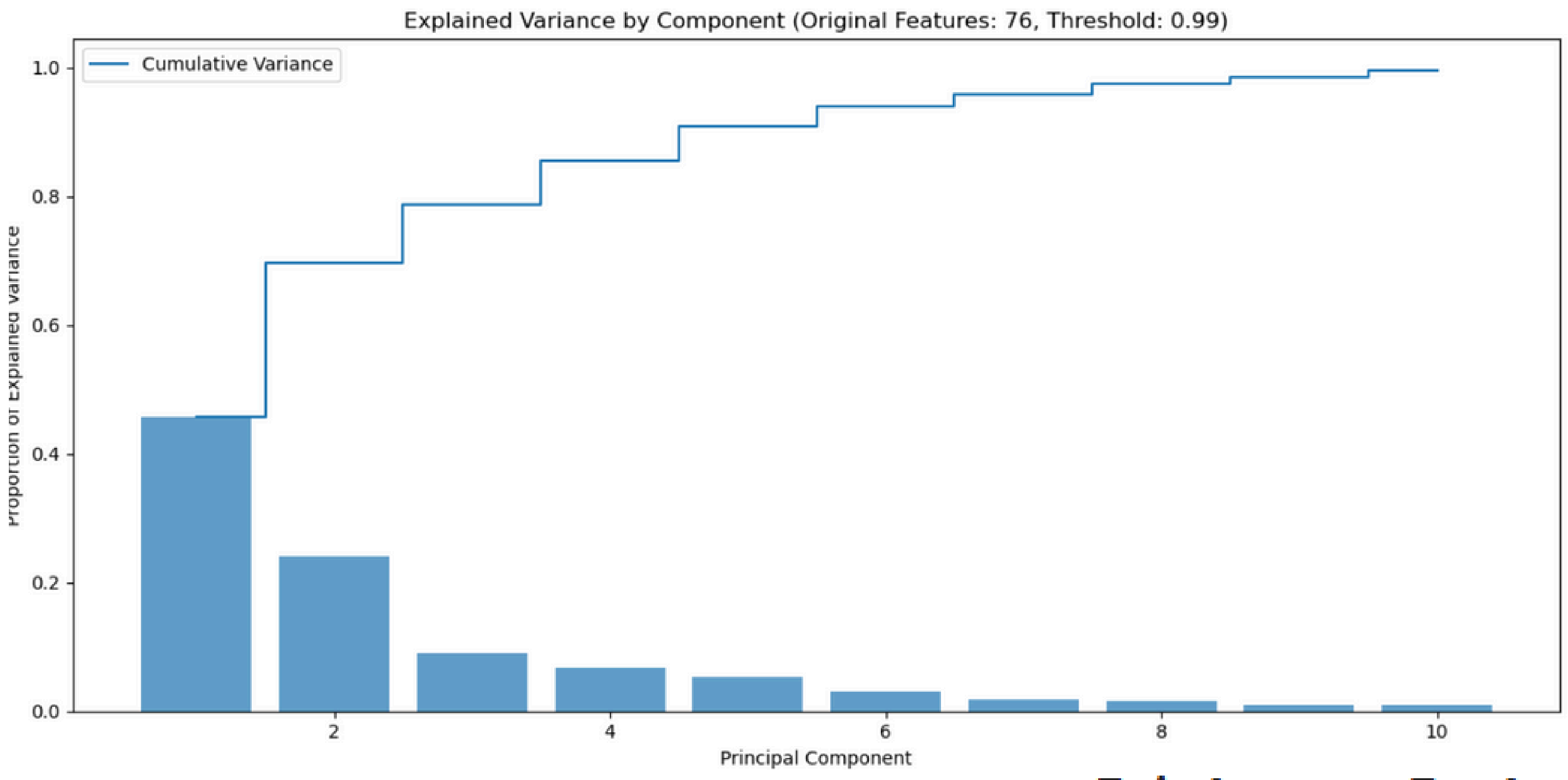
Dimensionality Reduction - PCA

Objective:

Understand how the model perform with a variable number of principal components, to simplify the problem by reducing the dimensions.

Analysis:

This analysis consists of adding PCA to the pipeline, after SMOTE and RUS, and analyse the results with different levels of explained variance equal to 0.99, 0.95 and 0.90.



Results:

- The number of principal components goes from 10 to 5 with the explained variance changing from 0.99 to 0.90.
- The performances remains high, with the recall showing a minimum decrease

Train Accuracy	Test Accuracy	Train Precision	Test Precision	Train Recall	Test Recall	Train F1	Test F1
0.999994	0.999994	1.0	1.0	0.999994	0.999994	0.999997	0.999997

Models Evaluation

Objective:

To confirm the expectation that indepently of the model, the classification task is simple with this dataset and to demonstrate that it's possible to have a correct classification even with a fewer number of features or dimensions.

The Compared Classifiers:

Logistic Regressor

The base classifier used, simple, well suited for binary classification problem, easy to understand, computationally efficient.

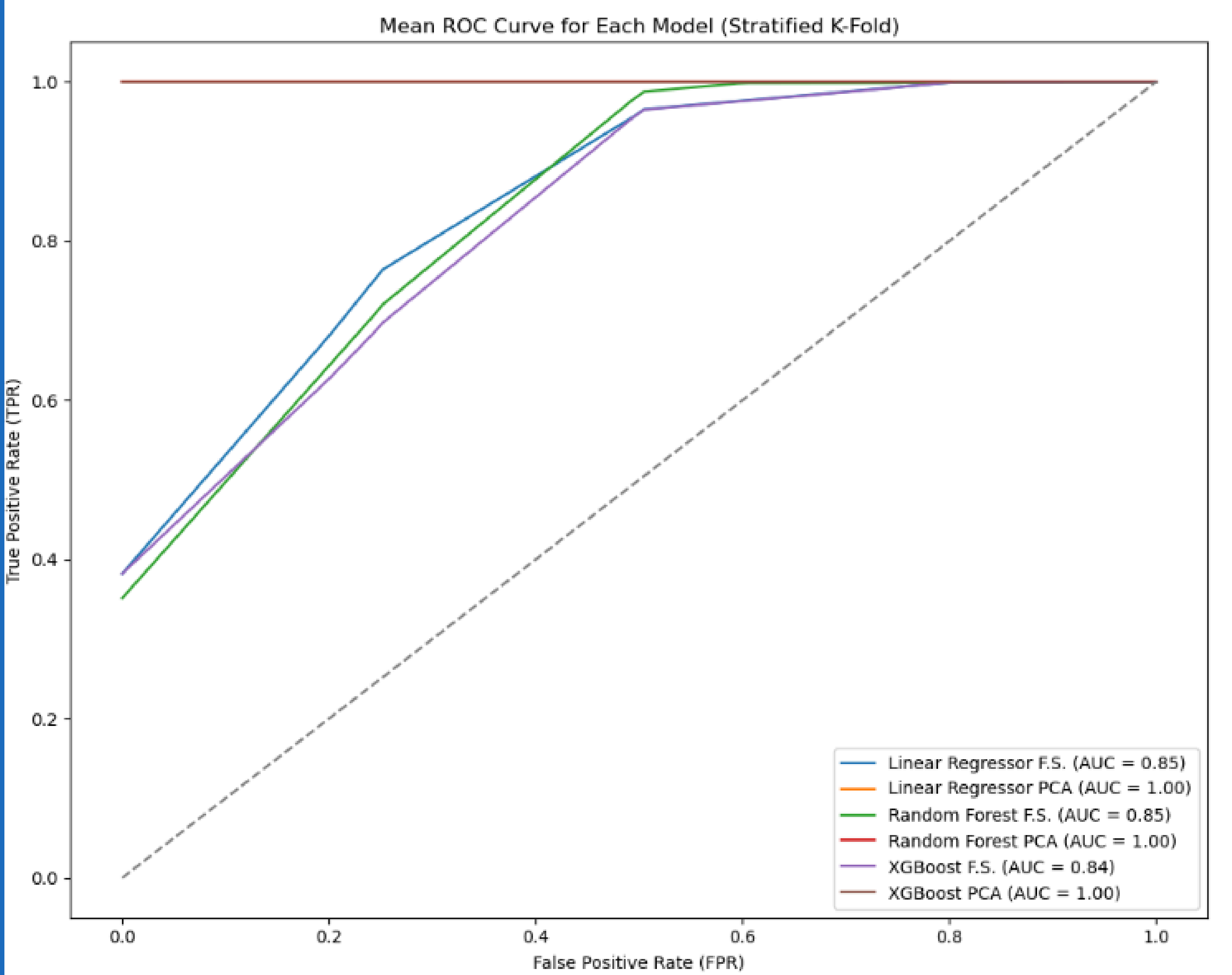
Random Forest

An ensemble classifier that builds multiple decision trees and combines their outputs for improved accuracy and robustness. It handles both classification and regression problems, is resistant to overfitting, and works well with high-dimensional datasets.

XGBoost

A gradient-boosting algorithm designed for efficiency and scalability. It's the most complex, it provides regularization to prevent overfitting, and is well-suited for both classification and regression tasks.

Models Evaluation



How to compare?

The classifiers are trained using all the different techniques seen so far, and from the score obtained using StratifiedKFold, the *Receiver Operating Characteristic* (ROC) curve has been computed

Results:

After the computation of the ROC curve, it's possible to notice that the models have similar performances on the data set.
The implemented approach, before the classification, is the only aspect responsible for the change in performances.

In fact, in the picture two distinct behaviors are observed. When the models are trained after applying PCA, the *Area Under the Curve, (AUC) is equal to 1*. Instead, when trained on the reduced feature set, AUC is equal to 0.85.

Final Considerations

About the goal

The setted goal of this project was to train a classifier to be able to correclty classify bad connections from the good ones, even whitout having a complete set of features.
This leads to firstly analyze the different rebalancing techniques and then to analyze feature selection and dimensionality reduction, adopting both feature importance and PCA to understand how the performance are affected, ending whit a confrontation of different classifiers.

Conclusions

1 **Rebalancing do not affect the performance**

The difference in values from istances of different classes is such that, even creating a lot of fictional istances, the distinction between the two remains clear and the performance are not affected.

2 **The model uses just few features to classify**

The model adopt just one or few features to take the decision. This is a remarkable aspect because it ensure that the model, even lacking of most of the feature, is able to correctly classify an unseen istance.

Final Considerations

Conclusions

3

Dimensions can be reduced even with PCA

The dimensionality can be effectively reduced using Principal Component Analysis (PCA) without substantially affecting performance. However, doing so eliminates the connections between features and the original data. Consequently, adopting PCA in a real-world context might not be ideal, as it would prevent the monitoring and removal of redundant features, which is otherwise possible

4

Other classifier behaves as well as the regressor

Given the simplicity of the problem with this dataset, other classifiers, even complex ones, can achieve similarly effective performance in the classification task

Future Implementations

Real-Time Validator

One possible feature implementations involves the creation of a real-time validator of connections by training the model with different dataset, representing different possible attacks and use it for the protection of critical resources. In a real-time context the model may not be provided with all of the features that are represented in this dataset, but it has been proven that, even with a small subset of features, it is still able to correctly classify all the unseen instances

The use of XAI

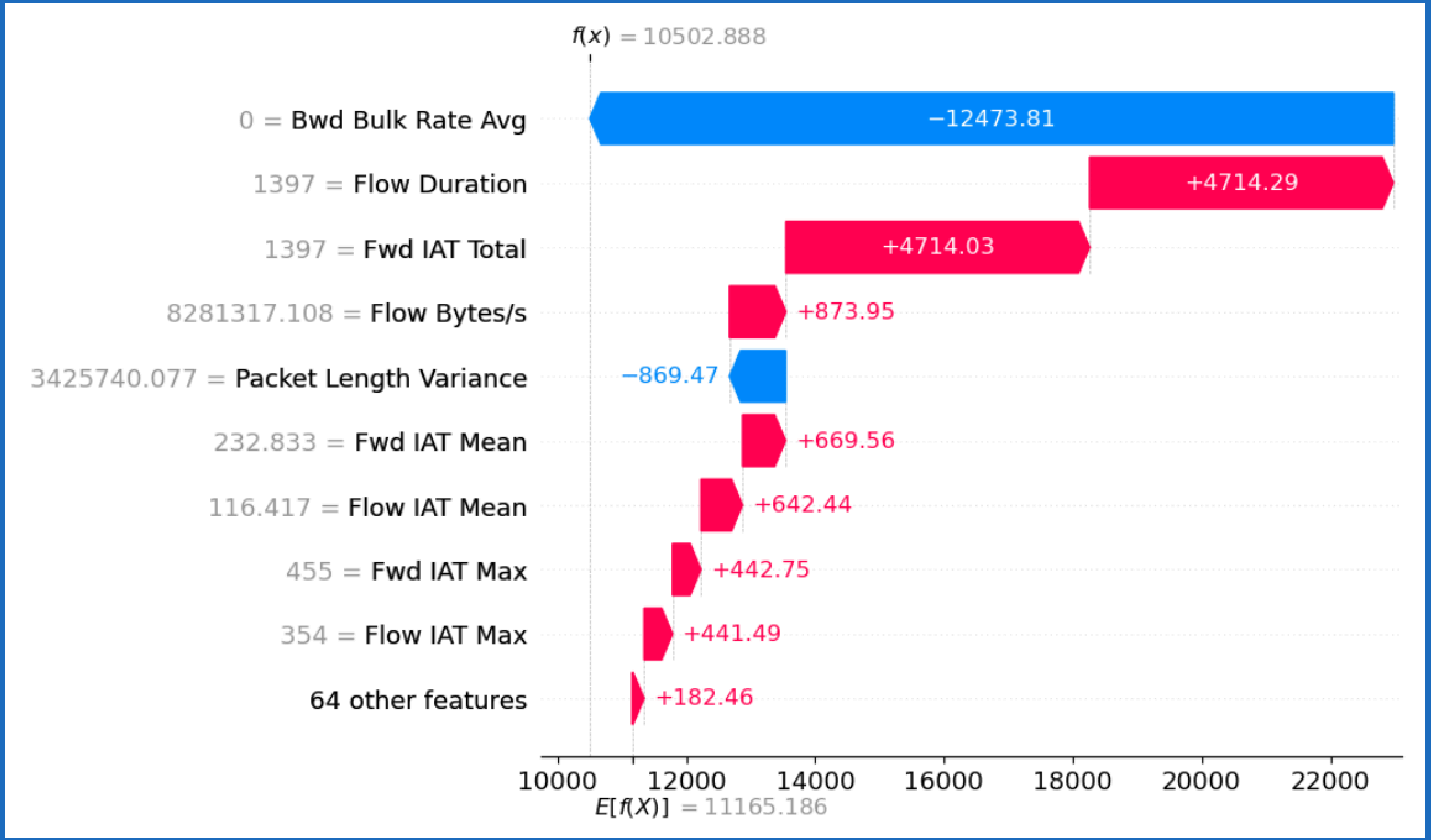
Adoption of models for the ‘***eXplainable Artificial Intelligence’*** can be usefull for:

- Obtain an explanation of how the model makes decisions
- Identify some key features that are crucial for the process
- Provides the real-time validator with at least those key features to secure the connections

An example of the use of SHAP is presented in the next slide

SHAP example

Waterfall Plot



The analysis with SHAP reveals that certain features, such as Bwd Bulk Rate Avg, Flow Duration, and Fwd IAT Total, are crucial for the model, as they contribute most significantly to the predictions. Therefore, their inclusion is essential to maintain high performance and to provide valuable insights for interpretation

FABBRI ANDREA

THANKS FOR THE ATTENTION

References

- ADASYN - https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.ADASYN.html
- Shapiro Wilk test - https://www.agnesevardanega.eu/wiki/r/test_statistici/shapiro-test